

cineMax – Manuale tecnico, Necchio Arianna, 16/09/2026, versione 2.1

- Indice

Installazione

- [Requisiti di sistema](#)
- [Setup ambiente](#)
- [Installazione programma](#)

Esecuzione ed uso

- [Setup e lancio](#)
- [Uso delle funzionalità](#)
 - [Classi dell'applicazione](#)
 - [Utente](#)
 - [Login](#)
 - [Proiezione](#)
 - [Prenotazione](#)
 - [UtenteNonEsistenteException](#)
 - [ProiezioneNonEsistenteException](#)
 - [menuCliente](#)
 - [menuBigliettaio](#)
 - [menuProiezionista](#)
 - [cineMax](#)
- [Dataset di test](#)
 - File ["Utenti"](#)
 - File ["Proiezioni"](#)
 - File ["Prenotazioni"](#)

Limiti della soluzione sviluppata

Sitografia/bibliografia

- Installazione

- Requisiti di sistema

L'applicazione è stata sviluppata su computer con sistema operativo Windows e con 8 GB di RAM. Mi aspetto che giri anche su elaboratori con memoria RAM di capacità più ridotta oppure su sistemi operativi differenti, anche più datati.

- Setup ambiente

- Per eseguire l'applicazione occorre avere installata la Java Virtual Machine (JVM in seguito), il compilatore Java che permette di codificare le istruzioni dal linguaggio in alto livello in bytecode (linguaggio macchina).
- Per installare l'ambiente Java occorre recarsi all'indirizzo <https://www.oracle.com/it/java/technologies/downloads/> (sito web di Oracle), scegliere la versione di Java desiderata, scegliere il sistema operativo per cui scaricare l'ambiente e cliccare sul link associato.

- Scompattare il file eseguibile, recarsi nella directory “Java”, spostarsi nella cartella denominata “jdk-*numero_versione*” (ad esempio “jdk-26”), infine aprire la directory “bin”.
- Copiare il classpath della cartella “bin”
- A questo punto è necessario collocare l’ambiente Java nelle variabili d’ambiente del proprio calcolatore affinché l’applicazione venga eseguita: le variabili d’ambiente indicano al programma dove trovare i file necessari alla compilazione e all’esecuzione; in questo caso specifico, individuano le librerie utilizzate dal programma (package compresi nell’ambiente Java).
Recarsi nelle impostazioni del sistema operativo, spostarsi su “sistema” e cliccare su “informazioni sul sistema”, accedere alle impostazioni di sistema avanzate, aprire la scheda “Variabili d’ambiente” e collocarsi nella parte “Variabili di sistema”. Selezionare “Path”, cliccare su “Modifica” e infine su “Nuovo”. Incollare il percorso della cartella “bin” e salvare.

● Installazione programma

- Scaricare l’applicazione cineMax dall’indirizzo <https://github.com/Necchio765298/cinemax.git>. Cliccare poi sul pulsante “Code” e scegliere “Download ZIP”.
- Il file che viene scaricato contiene in formato compresso i file binari e l’eseguibile “cinemax.jar”, oltre ai file sorgente, i manuali, i data set di test e la documentazione generata (con il tool javadoc).

● Esecuzione ed uso

● Setup e lancio

- Per eseguire l’applicazione è necessario innanzitutto scompattare il file compresso e collocarsi all’interno della cartella radice, ovvero “cinemax”; cliccare poi all’interno della barra di navigazione del file system e digitare il comando “cmd” per avviare il prompt dei comandi.
- Digitando il comando `java -jar bin/cinemax.jar` verrà visualizzata la schermata di benvenuto dell’applicazione.

● Uso delle funzionalità

Classi dell’applicazione

L’applicazione si sviluppa su diverse classi. Una classe è un’unità di compilazione che contiene il codice sorgente necessario al funzionamento del programma.

Ciascuna classe contiene dei metodi in numero variabile: dal costruttore ai metodi che realizzano le funzioni specifiche dell’applicazione. Un metodo è una funzione che, ricevuti in ingresso dei parametri (alcuni metodi lavorano senza ricevere parametri), restituisce un oggetto (struttura dati record formata da uno o più campi) o un valore.

L’applicazione è composta da classi che calcolano funzioni di base (ad esempio permettono di effettuare il login alla piattaforma o creano oggetti di tipo Utente, Bigliettaio o Proiezionista, cioè gli oggetti su cui l’applicazione gira) e altre che, basandosi sugli oggetti appena menzionati, svolgono le funzioni necessarie al funzionamento complessivo (come effettuare prenotazioni, o visualizzare le proiezioni nel palinsesto).

Le classi che compongono l’applicazione “cinemax” sono:

- Utente;
- Login;
- Proiezione;
- Prenotazione;
- UtenteNonEsistenteException
- ProiezioneNonEsistenteException;

- menuCliente;
- menuBigliettaio;
- menuProiezionista;
- cineMax contenente il metodo main.

Classe Utente

Campi

La classe Utente identifica oggetti di tipo “Utente”, ovvero gli utenti che si sono registrati all’applicazione. È composta dai seguenti campi:

- ID;
- nome;
- cognome;
- username;
- password;
- dataNascita;
- domicilio;
- ruolo.

Costruttori

La classe presenta due costruttori: il primo crea oggetti di tipo Utente al fine di memorizzarli nel file “Utenti.csv”; il secondo si occupa di creare oggetti “Utente” (prelevati dal file) per essere utilizzati dai metodi della classe stessa e dalle altre classi.

In particolare, il primo costruttore si occupa di creare un oggetto di tipo “Utente” inizializzando i campi della classe: in particolare, il campo “ID”, che rappresenta il codice identificativo dell’utente, è inizializzato con un numero progressivo, assegnato dal metodo generalID() ogni volta che viene aggiunto in coda un utente al file “Utenti.csv”. È stato scelto in precedenza di utilizzare la combinazione di username e password per identificare un utente (versione 1.0) ma non si è rivelato un metodo abbastanza solido per via della presenza di utenti con medesime credenziali; inoltre la password viene subito cifrata con il metodo statico passwordHash() della classe stessa e il valore restituito è memorizzato al campo password.

Il secondo costruttore inizializza lo stesso campo “ID” (non ha generatori numerici che ne assegnano un valore come sopra) con il valore alla posizione relativa al codice identificativo letto dal file Utenti.csv; per quanto riguarda gli altri campi, così come il campo password, non presentano variazioni rispetto all’altro costruttore.

Metodo generalID()

Questo metodo si occupa di generare il codice identificativo di ogni nuovo utente che si registra alla piattaforma: in particolare, viene letto l’ID dell’ultimo utente registrato e viene incrementato di uno in modo tale che ognuno risulta tracciabile con un numero univoco e irripetibile perché il metodo si basa su una numerazione progressiva e non randomica (evitando così numeri ripetuti generati randomicamente, come era stato scelto in un’implementazione precedente del metodo stesso – versione 2.0).

Viene utilizzato un vettore di tipo “String” denominato “dati” che suddivide ogni stringa letta dal file “Utenti.csv” in elementi e viene preso in considerazione l’elemento “dati[0]”, ovvero il codice ID e in un istante successivo il valore di dati[0] è incrementato di uno.

Il metodo si basa sulla ricerca sequenziale, ovvero scorre l’intero file una riga alla volta, e, data l’implementazione, richiede una complessità temporale in criterio di costo uniforme di $O(n)$, dove n è il

numero di righe del file; in criterio di costo logaritmico richiede (in tempo) $O(n \log(n))$, con n il numero delle righe e $\log(n)$ la lunghezza in bit del valore del codice ID.

Metodo `getUtente()`

La classe `Utente` dispone del metodo `getUtente()` per ottenere un oggetto di tipo “`Utente`” fornendo come parametro il proprio codice ID. Questo metodo è stato implementato per facilitare la creazione di un `Utente`, evitando di dover manualmente fornire un valore a tutti i campi della classe al medesimo scopo di ottenere l’oggetto (richiamando quindi il costruttore).

Il metodo si basa sulla lettura riga per riga del file “`Utenti.csv`” e sulla creazione di un vettore “`dati`” di tipo “`String`”, rappresentato da ciascuna riga, le cui celle di memoria sono costituite dagli elementi della riga separati dal segno “`,`”.

La struttura dati “vettore” consente una migliore gestione dell’oggetto (gli elementi sono a sé stanti) rispetto alla semplice “stringa” (composta da un unico elemento, ovvero la riga intera).

Per ogni riga letta dal file, il metodo confronta l’elemento `dati[0]`, rappresentante il codice identificativo con l’ID passato in argomento. Se i due valori coincidono, viene creato un oggetto di tipo `Utente`.

È stato scelto di inizializzare il campo “`dataNascita`”, al momento della creazione dell’oggetto, con una data formattata secondo il pattern “`yyyy-MM-dd`” della classe [DateTimeFormatter](#), al fine di ottenere un formato della data di nascita uguale per tutti gli utenti registrati.

Nel caso non venga trovato l’ID ricercato, è lanciata un’eccezione di tipo “`UtenteNonEsistenteException`”.

Metodo `registraUtente()`

Il metodo `registraUtente()` si occupa, ricevuto un oggetto di tipo “`Utente`” come parametro, di fornirne una rappresentazione testuale e di memorizzarlo come una nuova riga aggiunta in coda al file “`Utenti.csv`”.

La rappresentazione testuale dell’utente è fornita dal metodo `toString()`.

Nel caso la registrazione fallisca, è lanciata un’eccezione di tipo “`UtenteNonEsistenteException`”.

Metodi “getter”

La classe `Utente` dispone dei seguenti metodi “getter” implementati per rendere visibili alle altre classi del package i valori dei campi di questa classe:

- `getID();`
- `getNome();`
- `getCognome();`
- `getUsername();`
- `getPassword();` ha il modificatore di visibilità dichiarato “`private`” in modo che la password, nonostante cifrata, resti inaccessibile alle altre classi per ragioni di sicurezza;
- `getDataNascita();`
- `getDomicilio();`
- `getRuolo();`

Metodo `toString()`

La classe dispone di un metodo per la creazione di una stringa che rappresenta un oggetto di tipo “`Utente`”: sia per la memorizzazione di un utente nel file, sia per la rappresentazione testuale di un utente già registrato. Il metodo stampa i valori dei campi. Per il pattern testuale si rimanda alla sezione “[Data set di test](#)”.

Metodo passwordHash()

Il metodo si occupa di cifrare le password utilizzando lo standard SHA-256, ritenuto più sicuro di SHA-1.

Per il fatto che l'algoritmo di cifratura SHA-256 genera una sequenza di byte, è stato scelto di utilizzare un vettore di byte "hashBytes" che memorizza i valori generati in output dall'algoritmo: la password in chiaro ricevuta in argomento viene dapprima tradotta in una sequenza di byte, per poi applicare SHA-256 e cifrarli. Questi valori sono scanditi da un ciclo "for-each" che si occupa di convertirli in una stringa di 64 caratteri esadecimali (il nome "SHA-256" indica infatti che il testo cifrato occupa 256 bit, ovvero 64 caratteri), rappresentanti la password cifrata.

Affinché la password cifrata venga memorizzata come stringa nel file, è utilizzato il metodo toHexString().

Il metodo passwordHash() codifica le password in modo irreversibile: dispone soltanto la funzione di codifica (non di decodifica) aumentando la sicurezza delle password stesse in confronto a un qualsiasi altro metodo di codifica reversibile.

Affinché la stringa finale sia composta sempre da 64 caratteri, il frammento di codice "hex.length() == 1" aggiunge uno "0" all'inizio della stringa nel caso in cui un byte convertito in stringa genera un singolo carattere esadecimale (ad es. f, lasciando l'output composto da 63 caratteri).

È stato scelto di gestire le eccezioni utilizzando un costrutto "try-catch" al fine di delegare un'eventuale eccezione sollevata al chiamante, evitando di bloccare in modo anomalo l'esecuzione dell'intero programma.

Classe Login

Campi

La classe Login istanzia oggetti di tipo Login caratterizzati dai seguenti campi:

- Username;
- Password;
- Hash.

Costruttore

Il costruttore, ricevendo come parametri "username" e "password" passati dall'utente, si occupa di creare un oggetto di tipo Login costituito dai campi "username" e, andando a cifrare la password servendosi del metodo [passwordHash\(\)](#) della classe Utente, dal campo "password".

Metodo toString()

Il metodo toString() della classe Login fornisce una stringa che contiene i valori dei campi "username" e "password". Non viene utilizzato dalla versione 2.1 ma è rimasto dall'implementazione precedente del progetto (versione 1.0). È stato scelto di mantenerlo per via di una possibile utilità in fase di sviluppo del software.

Metodi "getter"

I metodi "getter()" con modificatore di visibilità "public" sono stati implementati affinché le altre classi possano avere accesso ai campi della classe Login dichiarati "private"; l'unico metodo "getter()" con modificatore di visibilità "private" è getPassword(), che non permette alle altre classi di ottenere il valore del campo "password" (cifrato).

- getUsername();
- getPassword();
- getHash().

Metodo login()

La classe Login dispone di un metodo statico login() che restituisce un oggetto di tipo "Utente". Questo metodo si serve delle credenziali di accesso di un utente già registrato per istanziare un oggetto di tipo Utente, il quale verrà salvato nel metodo [main\(\)](#) tramite una variabile globale che punterà all'oggetto in questione, in modo tale che nei metodi che richiedono tra i parametri formali un Utente, quest'ultimo venga passato senza la necessità che l'utente si identifichi ogni qual volta desideri compiere un'azione tra quelle disponibili per il proprio ruolo.

login() è collegato al file di testo "Utenti.csv"; legge riga per riga dal file confrontando username e password (memorizzati in un oggetto di tipo Login) passati in argomento con quelli memorizzati nel file. Il metodo si occupa, ad ogni riga letta, di dividerla (tramite il separatore di elementi ",") nella struttura dati vettore "dati" di tipo "String" in modo da fornire una posizione numerica agli elementi che lo compongono: "username" e "password" memorizzati nel file si trovano rispettivamente alle posizioni dati[3] e dati[4]. Per quanto riguarda il confronto delle password, il metodo confronta le password cifrate mentre la valutazione della coincidenza degli username avviene in chiaro.

La complessità computazionale di questo metodo è $O(n)$: la ricerca di "username" e "password" corrispondenti all'utente che intende accedere alla piattaforma richiede che vengano lette tutte le righe del file (n è il numero delle righe) fino ad individuare quella interessata.

Se entrambi i valori forniti corrispondono con quelli letti dal file, il metodo si occupa di estrarre dal vettore l'elemento dati[0] (rappresentante il codice identificativo dell'utente) per fornirlo come parametro al metodo [getUtente\(\)](#) della classe Utente per creare un oggetto di tipo Utente, il quale verrà restituito al chiamante.

La scelta di dividere ogni stringa del file di testo in un vettore di elementi è stata effettuata per una migliore gestione dell'oggetto: è possibile in questo modo ottenere un oggetto di tipo Utente senza istanziare ad ogni metodo chiamato tutti i campi ma inizializzandone solo uno (codice identificativo) per ottenere l'oggetto (tramite il metodo getUtente()).

Il metodo intercetta delle possibili eccezioni che si possono sollevare durante l'esecuzione:

- [UtenteNonEsistenteException](#) creata appositamente per eventuali utenti inesistenti a cui il metodo cerca di accedere;
- [IOException](#) nel caso il file "Utenti.csv" non sia accessibile;
- [Exception](#) per intercettare qualsiasi altro tipo di eccezione che non rientri nelle prime due.

Classe Proiezione

Campi

La classe Proiezione identifica oggetti di tipo "Proiezione", ovvero le proiezioni che vengono proiettate.

È composta dai seguenti campi:

- dataOra: identifica la data e l'ora della proiezione. Questo campo è stato scelto come "identificativo" della proiezione per il fatto che in un particolare giorno e orario una sola proiezione viene riprodotta;
- titolo;
- genere;
- regista;
- anno;
- durataMinuti;
- etaMinima;

- prezzoBiglietto.

Costruttore

La classe dispone di un solo costruttore che istanzia oggetti di tipo "Proiezione".

Metodo getProiezione()

Il metodo getProiezione() è stato ideato come "sostitutivo" del costruttore per ottenere oggetti di tipo "Proiezione". Piuttosto che inizializzare tutti i campi della classe per poi richiamare il costruttore, questo metodo riceve come unico parametro la data di una proiezione, cercando sequenzialmente nel file "Proiezioni.csv" a cui è collegato attraverso gli stream, e restituendo una sola proiezione avente la data ricevuta in argomento.

Il metodo si basa sulla struttura dati vettore "dati" di tipo "String" che divide ogni stringa letta dal file "Proiezioni.csv" in elementi ogniqualvolta viene incontrato il segno di interpunzione ",". Il metodo confronta la data di ogni proiezione memorizzata (identificata dall'elemento dati[0]) con la data ricevuta in argomento: se le date coincidono, la ricerca viene immediatamente interrotta e l'istruzione "return" restituisce al chiamante l'oggetto; altrimenti è stampato a video un opportuno messaggio d'errore e viene sollevata l'eccezione [ProiezioneNonEsistenteException](#).

Il metodo, scorrendo le righe del file fino a trovare quella interessata, impiega un tempo di computazione il cui ordine di grandezza è lineare ($O(n)$), dove n è il numero di righe lette precedenti a quella desiderata).

Metodo cercaProiezione()

Il metodo cercaProiezione() si occupa di creare una lista di proiezioni selezionate dal file "Proiezioni.csv" secondo un valore fornito dall'utente a uno dei criteri di ricerca disponibili, posizionati a livello dei parametri formali.

La ricerca avviene ispezionando sequenzialmente (riga per riga) il file sopra citato; un vettore "dati" di tipo "String" suddivide per "," ogni stringa del file.

Il metodo si basa su un confronto effettuato tramite lazy evaluation con operatore logico "or", al fine di individuare una corrispondenza tra un elemento del vettore e uno dei parametri. La lazy evaluation è stata scelta al posto del tradizionale "or" (che confronta entrambi gli operandi, anche se il primo ha come valore booleano "true"), al fine di evitare i confronti tra elementi successivi quando la corrispondenza è già stata individuata. Ciò permette di risparmiare tempo di computazione scorrendo in minor tempo tutte le voci del file.

Se il risultato booleano del blocco "if" è soddisfatto (il risultato dei confronti produce "true") è creato un nuovo oggetto di tipo "Proiezione" chiamando il costruttore e non il metodo getProiezione() perché si andrebbe a incrementare notevolmente il tempo di computazione del metodo stesso: sia cercaProiezione() che getProiezione() si basano sulla ricerca effettuata in tempo $O(n)$, dove n è il numero di righe del file "Proiezioni.csv" (ogni riga corrisponde a una proiezione). Dovendo quindi questo metodo con complessità temporale $O(n)$ richiamare il metodo getProiezione() anch'esso con la medesima complessità, si otterrebbe il risultato della ricerca dopo aver effettuato operazioni con ordine di grandezza quadratico, cioè $O(n^2)$. Richiamando invece il costruttore si mantiene l'ordine di grandezza lineare ($O(n)$), ottenendo, seppur non con la complessità temporale migliore, un risultato che non richiede tempi di ricerca oltre l'ordine di grandezza lineare.

Il metodo memorizza le proiezioni selezionate in una struttura dati lista, ovvero una sequenza di elementi tutti dello stesso tipo memorizzati nella variabile "memo" di tipo "ArrayList".

La lista è stata creata al fine di essere passata in argomento al metodo visualizzaProiezione() che si occupa di stampare gli elementi di "memo".

La lista è una struttura dati che offre la memorizzazione di più elementi: vista la necessità di fornire all'utente non una sola, ma più proiezioni aventi il medesimo criterio di ricerca, si è optato per questa struttura dati al posto di creare un singolo oggetto di tipo "Proiezione" che restituisce soltanto una voce tra tante richieste.

Metodo cercaProiezione() per intervallo di date

Al fine di semplificare l'esecuzione della funzionalità di ricerca delle proiezioni, è stato scelto di tenere separata la ricerca per intervallo di date dalla ricerca per qualsiasi altro criterio.

Questo override di cercaProiezione() utilizza anch'esso una struttura dati vettore, denominata "dati", di tipo String per suddividere in elementi le stringhe del file "Proiezioni.csv" e, per ogni data associata a ciascuna proiezione, controlla se rientra nell'intervallo di date, i cui estremi sono le date di inizio e fine fornite in argomento al metodo.

Come sopra, l'ordine di grandezza temporale della ricerca è mantenuto lineare scegliendo di richiamare anche qui il costruttore al posto di getProiezione() per creare gli oggetti da aggiungere alla lista sotto descritta.

Nel caso la data rientri nell'intervallo specificato, viene aggiunta in coda ad una struttura dati lista denominata, come sopra, "memo", di tipo "ArrayList". La lista consente di memorizzare più proiezioni aventi la data compresa nel periodo temporale desiderato; un solo oggetto di tipo Proiezione non avrebbe consentito di ottenere un elenco formato da più voci.

Metodo visualizzaProiezione()

Il metodo visualizzaProiezione si serve di un ciclo "for-each" che permette la visualizzazione delle proiezioni memorizzate nella lista "memo". È stato scelto di utilizzare questa variante del ciclo "for" perché non è possibile stabilire a priori la lunghezza della lista (e quindi indicare allo schema di composizione di iterazione il numero esatto di oggetti di tipo "Proiezione" da visualizzare) e di conseguenza il metodo ripete un numero indefinito di iterazioni fino a scorrere interamente "memo".

Metodi "getter()"

- getDataOra();
- getTitolo();
- getGenere();
- getRegista();
- getAnno();
- getDurataMinuti();
- getEtaMinima();

Metodi "setter()"

- setDataOra();
- setPrezzoBiglietto();

Metodo toString()

Il metodo toString() fornisce una rappresentazione testuale della Proiezione, sia per mostrarla all'utente, sia per essere aggiunta in coda al file "Proiezioni.csv" come una nuova proiezione.

Il metodo stampa a video i valori dei campi della classe. Il primo campo a essere visualizzato è la data della proiezione: è stato scelto di formattare la data secondo il pattern "yyyy-MM-dd HH:mm:ss" affinché il formato coincida con quello delle altre proiezioni già presenti nel file. Per il pattern testuale si rimanda alla sezione "[Data set di test](#)".

Metodo equals()

Il metodo equals() è stato implementato appositamente per confrontare tutti i valori dei campi di due oggetti di tipo "Proiezione" (uno ricevuto in argomento mentre l'altro esegue il metodo), al fine di stabilirne l'uguaglianza. È stato scelto di verificare la corrispondenza di tutti i valori dei campi degli oggetti da confrontare per verificare la coincidenza di tutti quanti contemporaneamente tramite l'operatore booleano "==" (ad esempio il frammento di codice "p.getDataOra()==this.dataOra" confronta la data – "p" è l'oggetto ricevuto in argomento). Nel caso tutti i valori dei campi degli oggetti comparati siano uguali, il metodo stabilisce "true" la corrispondenza, "false" altrimenti.

equals() è utilizzato nei metodi [modificaProiezione\(\)](#) e [eliminaProiezione\(\)](#).

Classe Prenotazione

La classe Prenotazione è dedicata alla creazione di oggetti di tipo "Prenotazione", ovvero le prenotazioni che gli utenti possono creare, visualizzare, modificare e eliminare.

Campi

La classe è composta dai seguenti campi:

- codice: è un campo il cui valore viene associato al momento della creazione e del recupero di una prenotazione;
- utente;
- proiezione;
- numeroBiglietto.

Costruttori

La classe dispone di due costruttori che si occupano: uno di istanziare oggetti di tipo "Prenotazione" al fine di essere convertiti in stringa (tramite il metodo toString()) e memorizzati nel file "Prenotazioni.csv"; l'altro si occupa del processo inverso, ovvero preleva una stringa dal file e la converte in un oggetto di tipo "Prenotazione".

I costruttori della classe si occupano di creare oggetti unendo gli oggetti "Utente" e "Proiezione", a cui si aggiungono un codice identificativo e il numero dei biglietti prenotati. In questo modo l'oggetto Prenotazione identifica una specifica proiezione che un certo utente vuole visualizzare, il numero dei biglietti che ha acquistato e un codice che identifica la prenotazione stessa.

Nello specifico, un costruttore si occupa di istanziare un nuovo oggetto andando ad assegnare al campo "codice" un valore randomico; affinché nella fase di recupero di una prenotazione tale campo non venga sovrascritto da un nuovo valore generato casualmente (quindi si perderebbe definitivamente il valore del codice che la identifica), è stato scelto di utilizzare un secondo costruttore che inizializza il valore del campo "codice" andandolo a leggere dal file "Prenotazione.csv".

Metodo registraPrenotazione()

Il metodo registraPrenotazione() si occupa di memorizzare una prenotazione creata nel file "Prenotazioni.csv". La prenotazione che si intende registrare è convertita in stringa (tramite il metodo toString()), ovvero il formato testuale con cui è possibile procedere al salvataggio su file; altrimenti non sarebbe possibile memorizzare l'oggetto avente come tipo "Prenotazione".

È stato scelto di formattare la stringa (ovvero quali informazioni scrivere su file) nella seguente composizione:

- data della prenotazione: è la data formattata secondo il pattern “yyyy-MM-dd HH-mm-ss” della classe [LocalDateTime](#) per avere congruenza tra le date delle prenotazioni e quelle delle proiezioni; la data identifica univocamente la proiezione che l’utente intende visionare;
- codice identificativo dell’utente: si risale facilmente all’utente che ha prenotato per una specifica proiezione;
- nome e cognome dell’utente: permettono una più rapida visualizzazione dell’utente che ha prenotato; non sono considerati un riferimento assoluto per il fatto che possono esistere uno o più utenti con lo stesso nome e cognome, mentre il codice identificativo distingue un utente dall’altro, sebbene siano omonimi;
- titolo della proiezione: anche il titolo, come per il nome e il cognome, fornisce un rapido riferimento alla proiezione ma, essendoci la possibilità di più proiezioni omonime, la data è l’“identificativo” di una proiezione specifica;
- numero biglietti: quest’informazione fornisce il numero dei biglietti che un utente ha acquistato per una prenotazione;
- codice: è il formato testuale del codice generato randomicamente nel momento in cui viene generata una nuova prenotazione da memorizzare su file.

Metodi “getter”

- `getCodice();`
- `getUtente();`
- `getProiezione();`
- `getCostoTotale();`
- `getTotaleBiglietti();` il metodo si basa su un vettore denominato “dati” di tipo “String” che suddivide ciascuna riga del file “Prenotazioni.csv” in elementi separati da “,”. Questo consente di andare a confrontare l’elemento `dati[0]`, ovvero la data di ogni prenotazione con la data della prenotazione di cui si vuole ottenere il numero dei biglietti.

Sono state scelte le date per effettuare il confronto perché sono gli “identificativi” delle proiezioni a cui è associata una prenotazione. Una volta individuata la prenotazione di cui calcolare il totale dei biglietti, il metodo ne restituisce la somma.

Questo metodo è utilizzato per controllare se il numero dei biglietti per una proiezione è minore del numero dei posti disponibili nella sala oppure ne eccede la capacità.

Metodi “setter”

- `setProiezione();`
- `setNumeroBiglietto();`

Metodo `toString()`

La classe Prenotazione dispone di un metodo `toString()`: fornisce una rappresentazione testuale di una prenotazione appena creata da memorizzare nel file “Prenotazioni.csv”, fornendo tra il valore dei campi della classe anche il valore randomico del campo “codice”; permette inoltre di visualizzare una prenotazione esistente, ricavata dal file, mostrando il valore di tutti i campi. Per il pattern testuale si rimanda alla sezione “[Data set di test](#)”

Metodo `equals()`

Il metodo `equals()` è stato implementato appositamente per confrontare tutti i valori dei campi di due oggetti di tipo “Prenotazione” (uno ricevuto in argomento mentre l’altro esegue il metodo), al fine di stabilirne l’uguaglianza. È stato scelto di verificare la corrispondenza di tutti i valori dei campi degli oggetti da confrontare per verificare la coincidenza di tutti quanti contemporaneamente tramite l’operatore booleano “==” (ad esempio, il frammento di codice “`p.getCodice()==this.codice`” confronta il codice di

due oggetti – “p” è l’oggetto di tipo “Proiezione” che esegue il metodo). Nel caso tutti i valori dei campi degli oggetti comparati siano uguali, il metodo stabilisce “true” la corrispondenza, “false” altrimenti.

equals() è utilizzato nei metodi [modificaPrenotazione\(\)](#) e [eliminaPrenotazione\(\)](#).

Classe UtenteNonEsistenteException

La classe UtenteNonEsistenteException è stata implementata per sollevare eccezioni che confermano l’inesistenza di un particolare utente ricercato tramite il codice identificativo associato.

Costruttore

Il costruttore, ricevendo un codice identificativo di tipo long come parametro, stampa a video un messaggio che avverte dell’inesistenza di quel particolare utente ricercato.

Classe ProiezioneNonEsistenteException

La classe ProiezioneNonEsistenteException è stata implementata per sollevare eccezioni apposite per gestire le proiezioni inesistenti a cui si tenta di accedere.

Costruttori

La classe dispone di due costruttori:

Il primo costruttore, ricevendo un parametro di tipo “LocalDateTime” rappresentante la data che identifica una e una sola proiezione, stampa a video un opportuno messaggio d’errore nel caso in cui la proiezione ricercata non esiste, informando l’utente dell’inesistenza della stessa.

Il secondo costruttore, riceve un parametro booleano che valuta o meno l’esistenza di una proiezione ricercata. È stato scelto di implementare questo particolare costruttore per evitare di scrivere più volte nel metodo [main\(\)](#) un messaggio che informa l’utente dell’inesistenza della proiezione ricercata con un valore di uno specifico parametro tramite il metodo [cercaProiezione\(\)](#).

Entrambe le classi estendono direttamente la classe [Exception](#), adattando meglio le eccezioni sollevate al contesto in cui vengono sollevate.

Classe menuCliente

La classe menuCliente dispone di funzionalità apposite per gli utenti registrati all’applicazione come “clienti”. In particolare, consente agli utenti di avvalersi delle seguenti funzionalità:

- visualizzaPrenotazione();
- creaPrenotazione();
- modificaPrenotazione();
- eliminaPrenotazione();

Metodo visualizzaPrenotazione()

Il metodo visualizzaPrenotazione() si occupa di stampare a video tutti gli oggetti di tipo “Prenotazione” memorizzati nella lista creata tramite [cercaPrenotazione\(\)](#)

È utilizzato un ciclo “for-each” che, vista la necessità di stampare a video il contenuto della lista memorizzante un numero variabile di oggetti di tipo “Prenotazione”, non richiede che venga fornita a priori la sua lunghezza: ciò permette di visualizzare gli elementi di liste di lunghezza variabile.

È utilizzato il metodo [toString\(\)](#) della classe Prenotazione per fornire una rappresentazione testuale dell’oggetto, altrimenti non sarebbe stato possibile visualizzarlo.

Il tutto è inserito in un costrutto “try-catch” per intercettare eventuali eccezioni che si possono sollevare.

Metodo creaPrenotazione()

Il metodo creaPrenotazione permette agli utenti di creare un nuovo oggetto di tipo "Prenotazione" da memorizzare su file "Prenotazioni.csv", fornendolo in argomento al metodo [registraPrenotazione\(\)](#) della classe Prenotazione.

Al fine di creare l'oggetto, il metodo si occupa di istanziare i campi del costruttore (per poi richiamarlo al fine di ottenere l'oggetto del tipo di cui sopra) della classe Prenotazione: fornisce quindi l'utente, acquisito dal metodo [main\(\)](#) al momento del login (vedasi metodo [login\(\)](#) della classe Login), e al contempo lo mostra a video; crea un oggetto di tipo "Proiezione" grazie al metodo [getProiezione\(\)](#), a cui è fornito l'orario della stessa (ricevuto in argomento); riceve da input, chiedendo l'inserimento all'utente, il numero di biglietti da acquistare, memorizzati come numero intero.

Il metodo si occupa inoltre di avvertire l'utente nel caso il numero dei biglietti ecceda la capacità della sala: il computo è affidato al metodo [getTotaleBiglietti\(\)](#) della classe Prenotazione; infine è richiamato il metodo [getCodice\(\)](#), per fornire all'utente il codice della prenotazione.

Il tutto è inserito in un costrutto "try-catch" che intercetta eccezioni eventuali che si possono sollevare durante l'esecuzione del metodo: è stato scelto di stampare a video, tramite il metodo [getMessage\(\)](#) l'errore verificatosi, per comprendere durante la fase implementativa la natura del malfunzionamento e attuare una correzione mirata.

Metodo modificaPrenotazione()

Il metodo modificaPrenotazione() permette di modificare una stringa (prenotazione) all'interno del file "Prenotazioni.csv" solo se le date delle prenotazioni (prenotazione in memoria e da memorizzare) siano successive alla data odierna (calcolata con [now\(\)](#)): questo confronto è stato effettuato grazie ai metodi [isBefore\(\)](#) e [isAfter\(\)](#) della classe [LocalDateTime](#) che stimano se le date ricevute in argomento al metodo modificaPrenotazione() soddisfino entrambe la condizione.

La funzione di modifica è stata implementata basandosi in totale su due file di tipo ".csv": il primo è "Prenotazioni.csv" a cui è collegato uno stream per la lettura delle voci (legge riga per riga finché non viene individuata quella da modificare); il secondo è un file temporaneo (denominato con "pre_numeri.csv", che verrà poi rinominato con il nome del file sorgente, sul quale vengono ricopiate, tramite uno stream collegato in scrittura, le voci dell'altro file a meno di quella interessata, la quale viene sostituita con una stringa (che era, prima di essere convertita in testo, un oggetto di tipo "Prenotazione").

Al termine della scrittura, il file temporaneo viene rinominato in "Prenotazioni.csv" mentre il corrispondente sorgente viene eliminato.

Purtroppo non è stato possibile sviluppare una soluzione più efficace, collegando al codice (tramite stream) un solo file.

Il tutto è inserito in un costrutto "try-catch" che intercetta eventuali eccezioni che si possono sollevare nel caso gli stream non si colleghino al file oppure quest'ultimo è utilizzato dal SO; mentre la clausola "finally" chiude gli stream, sia in caso di avvenuta modifica, che in caso di errori, sia le date delle prenotazioni non soddisfino la condizione sopra descritta.

All'altezza di ciascun blocco "catch" è stato scelto di stampare a video l'eventuale eccezione che viene sollevata con l'istruzione "System.err.println()", sviluppata appositamente con il campo "err" per mostrare gli errori verificatisi in corso d'esecuzione.

Un ulteriore blocco "try-catch" segue le istruzioni appena descritte: al suo interno vi sono due istruzioni "if", le quali consentono di rinominare il file temporaneo solo se il sorgente viene eliminato.

Nel blocco "try-catch" primario, ovvero le istruzioni atte alla modifica, vengono istanziati gli oggetti di tipo "File", creati all'inizio del metodo (quali la rappresentazione astratta del file temporaneo e del sorgente);

successivamente è cercata per data e ora la prenotazione da modificare, tratta dalla lista di tipo `ArrayList` ed estratta dalla testa con il metodo `get()`.
Dopodiché viene richiamato il metodo `creaPrenotazione()` per creare un nuovo oggetto di tipo "Prenotazione", ovvero la prenotazione che andrà in sostituzione di quella appena trovata.
Viene poi creato il vettore "dati" di tipo "String" per la suddivisione in elementi delle stringhe lette dal file: di ciascuna Prenotazione memorizzata in "Prenotazioni.csv" si traggono i valori degli elementi del vettore che serviranno poi ai metodi `getUtente()` e `getProiezione()` per creare oggetti di tipo "Utente" e "Proiezione", oltre agli elementi `dati[5]` e `dati[6]` rappresentanti rispettivamente il numero dei biglietti e il codice, tutti atti ad istanziare un nuovo oggetto di tipo "Prenotazione" chiamando il costruttore e fornendo questi valori ai parametri formali. Quest'ultimo oggetto è creato ex novo ad ogni iterazione del ciclo "while" che consente la lettura di tutte le righe del file e corrisponde alla prenotazione contenuta in ciascuna linea. Viene utilizzato il metodo `equals()` della classe Prenotazione per stabilire se la prenotazione da modificare (in precedenza sita in testa alla lista) coincida con quella letta dal file: in caso affermativo, viene sostituita con la prenotazione appena creata con `creaPrenotazione()` (ovvero viene scritta sul file temporaneo); al contrario, viene ricopiata la prenotazione dal file sorgente.

Metodo `eliminaPrenotazione()`

Il metodo `eliminaPrenotazione()` permette di eliminare una stinga (prenotazione) all'interno del file "Prenotazioni.csv" solo se la data della prenotazione in memoria sia successiva alla data odierna (calcolata con `now()`): questo confronto è stato effettuato grazie al metodo `compareTo()` della classe `LocalDateTime` che stima se la data ricevuta in argomento al metodo `modificaPrenotazione()` sia successiva o precedente alla data odierna.

Questa funzione è stata implementata basandosi in totale su due file di tipo ".csv": il primo è "Prenotazioni.csv" a cui è collegato uno stream per la lettura delle voci (legge riga per riga finché non viene individuata quella da eliminare); il secondo è un file temporaneo (denominato con "pre_numeri.csv", che verrà poi rinominato con il nome del file sorgente, sul quale vengono ricopiate, tramite uno stream collegato in scrittura, le voci dell'altro file a meno di quella interessata, la quale viene sostituita con una stringa (che era, prima di essere convertita in testo, un oggetto di tipo "Prenotazione").

Al termine della scrittura, il file temporaneo viene rinominato in "Prenotazioni.csv" mentre il corrispondente sorgente viene eliminato.

Purtroppo non è stato possibile sviluppare una soluzione più efficace, collegando al codice (tramite stream) un solo file.

Il tutto è inserito in un costrutto "try-catch" che intercetta eventuali eccezioni che si possono sollevare nel caso gli stream non si colleghino al file oppure quest'ultimo è utilizzato dal SO; mentre la clausola "finally" chiude gli stream, sia in caso di avvenuta eliminazione, che in caso di errori, sia la data della prenotazione non soddisfi la condizione sopra descritta.

All'altezza di ciascun blocco "catch" è stato scelto di stampare a video l'eventuale eccezione che viene sollevata con l'istruzione "System.err.println()", sviluppata appositamente con il campo "err" per mostrare gli errori verificatisi in corso d'esecuzione.

Un ulteriore blocco "try-catch" segue le istruzioni appena descritte: al suo interno vi sono due istruzioni "if", le quali consentono di rinominare il file temporaneo solo se il sorgente viene eliminato.

Nel blocco "try-catch" primario, ovvero le istruzioni atte alla modifica, vengono istanziati gli oggetti di tipo "File", creati all'inizio del metodo (quali la rappresentazione astratta del file temporaneo e del sorgente); successivamente è cercata per data, ora e codice la prenotazione da eliminare, tratta dalla lista di tipo `ArrayList` ed estratta dalla testa con il metodo `get()`.

Viene poi creato il vettore "dati" di tipo "String" per la suddivisione in elementi delle stringhe lette dal file: di ciascuna Prenotazione memorizzata in "Prenotazioni.csv" si traggono i valori degli elementi del vettore

che serviranno poi ai metodi [getUtente\(\)](#) e [getProiezione\(\)](#) per creare oggetti di tipo "Utente" e "Proiezione", oltre agli elementi dati[5] e dati[6] rappresentanti rispettivamente il numero dei biglietti e il codice, tutti atti ad istanziare un nuovo oggetto di tipo "Prenotazione" chiamando il costruttore e fornendo questi valori ai parametri formali. Quest'ultimo oggetto è creato ex novo ad ogni iterazione del ciclo "while" che consente la lettura di tutte le righe del file e corrisponde alla prenotazione contenuta in ciascuna linea. Viene utilizzato il metodo [equals\(\)](#) della classe Prenotazione per stabilire se la prenotazione da eliminare (in precedenza sita in testa alla lista) coincida con quella letta dal file: in caso affermativo, viene inserita sul file destinazione un'interruzione di linea; al contrario, viene ricopiata la prenotazione dal file sorgente.

Classe menuBigliettaio

La classe menuBigliettaio è stata implementata per fornire le funzionalità di gestione delle prenotazioni necessarie ai bigliettaio.

Dispone di tre metodi:

- `cercaPrenotazione()` per la ricerca di una prenotazione per svariati criteri di ricerca;
- il suo override (per la ricerca con un intervallo di date);
- `visualizzaPrenotazione()` per la visualizzazione delle stesse in coda alla ricerca.

Metodo `cercaPrenotazione()`

Il metodo si occupa di ricercare una prenotazione ricevendo come criterio un valore tra i parametri ammessi.

La ricerca è effettuata nel file "Prenotazioni.csv" tramite un algoritmo di ricerca sequenziale: per ogni riga letta dal file, è stato deciso di dividerla in elementi, istanziando un vettore "dati" di tipo "String", secondo il segno di interpunzione ",". Questo consente di confrontare (tramite lazy evaluation) ogni elemento del vettore con il corrispondente criterio di ricerca (parametro formale) inizializzato al momento della chiamata del metodo.

All'interno del blocco "if", qualora la condizione risulti soddisfatta, viene istanziato un nuovo oggetto di tipo "Prenotazione", utilizzando il costruttore della classe Prenotazione che utilizza, come valori da ricevere in argomento, due degli elementi del vettore, ovvero il codice e il numero dei biglietti, oltre agli oggetti di tipo "Utente" e "Proiezione" ricavati dai rispettivi e omonimi metodi "getter".

Ad ogni corrispondenza individuata, la prenotazione che viene creata è aggiunta in coda alla struttura dati lista, scelta per poter memorizzare più oggetti dello stesso tipo sotto il nome di "memo", al fine poi di essere stampati a video grazie al metodo `visualizzaPrenotazione()`.

È stato scelto di chiudere i flussi di caratteri (stream) all'interno della clausola "finally" del blocco "try-catch" per evitare che il metodo termini, sollevando un'eccezione, senza che i canali di comunicazione con il file rimangano aperti.

Questo metodo è stato implementato con le stesse scelte algoritmiche e architetturali di [cercaProiezione\(\)](#), metodo della classe Proiezione.

Metodo `cercaPrenotazione()` (intervallo di date)

L'override del metodo `cercaPrenotazione()` è stato strutturato come appena descritto sopra: fa uso quindi di un algoritmo di ricerca sequenziale che scorre l'intero file, avente una complessità computazionale $O(n)$; a sua volta richiama i metodi [getProiezione\(\)](#) e [getUtente\(\)](#), entrambi con complessità lineare $O(n)$.

Nel complesso il metodo ha un tempo di computazione con ordine di grandezza n^3 , dove n è la dimensione

(in righe) del file da cui sono prelevate le informazioni.

`cercaPrenotazione()` per intervallo di date inoltre istanzia una lista, creata tramite la classe [ArrayList](#), per la memorizzazione di più oggetti di tipo "Prenotazione" in modo da poter visualizzare, tramite il metodo [visualizzaPrenotazione\(\)](#) un elenco di prenotazioni (e quindi di oggetti), al posto di visualizzarne uno solo, ottenuto semplicemente tramite l'istruzione "return" che restituisce un singolo oggetto e non di una lista di oggetti.

Viene fatto uso anche del vettore che divide le stringhe lette dal file in elementi, per poterli meglio gestire, al posto di dover maneggiare ad ogni necessità l'intero oggetto (cioè tutti i campi) anche dove non sono necessari. La suddivisione in elementi permette quindi di estrapolare l'elemento interessato dalla stringa letta dal file, ovvero un particolare campo dell'oggetto.

Per quanto riguarda la condizione che stabilisce o meno l'aggiunta di una prenotazione alla lista, si valuta se le date di ciascuna prenotazione rientrano nell'intervallo temporale delimitato dalle date di inizio e fine ricevute in argomento.

In ultimo, anche la chiusura degli stream avviene nel blocco "finally" del "try-catch" affinché si abbia la certezza che i canali di comunicazione tra il metodo e il file vengano chiusi una volta terminata l'operazione.

Metodo `visualizzaPrenotazione()`

Il metodo `visualizzaPrenotazione()` si occupa di stampare a video tutti gli oggetti di tipo "Prenotazione" memorizzati nella lista creata tramite [cercaPrenotazione\(\)](#).

È utilizzato un ciclo "for-each" che, vista la necessità di stampare il contenuto della lista avente un numero variabile di oggetti, non richiede che venga fornita a priori la sua lunghezza: ciò permette di visualizzare gli elementi di liste di lunghezza variabile.

È utilizzato il metodo [toString\(\)](#) della classe `Prenotazione` per fornire una rappresentazione testuale dell'oggetto, altrimenti non sarebbe stato possibile visualizzarlo.

Classe `menuProiezionista`

La classe `menuProiezionista` è stata sviluppata per fornire funzionalità specifiche per gli utenti registrati all'applicazione come proiezionisti.

In particolare, comprende i seguenti metodi:

- `creaProiezione();`
- `aggiungiProiezione();`
- `modificaProiezione();`
- `eliminaProiezione();`

Metodo `creaProiezione()`

Il metodo `creaProiezione()` è stato implementato per creare una nuova proiezione che andrà ad aggiungersi, tramite il metodo `aggiungiProiezione()`, all'elenco delle proiezioni nel palinsesto; nello specifico, viene aggiunta in coda al file "Proiezioni.csv".

Il metodo consente all'utente, tramite la classe `Console`, di inserire manualmente i valori della proiezione che andranno ad inizializzare i campi della classe `Proiezione`. L'ultima istruzione del metodo, prima di stampare a video la proiezione tramite il metodo [toString\(\)](#), è la chiamata al [costruttore](#), necessaria alla creazione del nuovo oggetto).

Metodo aggiungiProiezione()

Questo metodo si occupa, tramite stream, di memorizzare una proiezione in coda al file "Proiezioni.csv" grazie all'argomento "true" che riceve il costruttore della classe [FileWriter](#).

Nel metodo, al fine di controllare se la proiezione che si intende aggiungere si accavalla con una esistente, è stato scelto di utilizzare un ciclo "while" per la lettura di ogni stringa componente il file, la quale è divisa in elementi grazie al vettore "dati" di tipo "String", che consente, preso in considerazione l'elemento dati[0] (data e ora della proiezione), di confrontare la data e l'ora di entrambe.

Se non viene individuata nessuna corrispondenza, il che significa che non vi siano già altre proiezioni in programma in quel giorno e a quella ora, l'oggetto di tipo "Proiezione" ricevuto in argomento viene memorizzato nel file.

Metodo modificaProiezione()

Il metodo modificaProiezione() permette di modificare una stringa (proiezione) all'interno del file "Proiezioni.csv".

La funzione di modifica è stata implementata basandosi in totale su due file di tipo ".csv": il primo è "Proiezioni.csv" a cui è collegato uno stream per la lettura delle voci (legge riga per riga finché non viene individuata quella da modificare); il secondo è un file temporaneo (denominato con "pro_numeri.csv", che verrà poi rinominato con il nome del file sorgente, sul quale vengono ricopiate, tramite uno stream collegato in scrittura, le voci dell'altro file a meno di quella interessata, la quale viene sostituita con una stringa (che era, prima di essere convertita in testo, un oggetto di tipo "Proiezione").

Al termine della scrittura, il file temporaneo viene rinominato in "Proiezioni.csv" mentre il corrispondente sorgente viene eliminato.

Purtroppo non è stato possibile sviluppare una soluzione più efficace, collegando al codice (tramite stream) un solo file.

Il tutto è inserito in un costrutto "try-catch" che intercetta eventuali eccezioni che si possono sollevare nel caso gli stream non si colleghino al file oppure quest'ultimo è utilizzato dal SO; mentre la clausola "finally" chiude gli stream, sia in caso di avvenuta modifica, che in caso di errori, sia esistano prenotazioni per la proiezione da modificare (verificato con il metodo isEmpty(), che stabilisce se la lista contenente oggetti di tipo "Prenotazione" sia vuota).

All'altezza di ciascun blocco "catch" è stato scelto di stampare a video l'eventuale eccezione che viene sollevata con l'istruzione "System.err.println()", sviluppata appositamente con il campo "err" per mostrare gli errori verificatisi in corso d'esecuzione.

Un ulteriore blocco "try-catch" segue le istruzioni appena descritte: al suo interno vi sono due istruzioni "if", le quali consentono di rinominare il file temporaneo solo se il sorgente viene eliminato.

Nel blocco "try-catch" primario, ovvero le istruzioni atte alla modifica, vengono istanziati gli oggetti di tipo "File", creati all'inizio del metodo (quali la rappresentazione astratta del file temporaneo e del sorgente); dopodiché viene richiamato il metodo [getProiezione\(\)](#) per ricercare la proiezione da modificare con il relativo orario specificato, il metodo creaProiezione() per creare la nuova proiezione in sostituzione di quella ottenuta e creati nuovi oggetti (alcuni di tipo "String", un "LocalDateTime" e un "double", oltre al vettore "dati" di tipo "String" per la suddivisione in elementi delle stringhe lette dal file "Proiezioni.csv") che serviranno al costruttore della classe Proiezione per istanziare un nuovo oggetto di tipo "Proiezione" grazie ai valori di "dati" ottenuti dalle righe del file. Quest'ultimo oggetto è creato ex novo ad ogni iterazione del ciclo "while" e corrisponde alla proiezione contenuta in ciascuna linea del file.

La suddivisione delle stringhe è preferita alla chiamata al metodo getProiezione() per mantenere il tempo di costruzione dell'oggetto lineare (la chiamata ad ogni iterazione del ciclo "while" porterebbe il tempo di computazione da $O(n)$, con n il numero di iterazioni, a $O(n^2)$, in cui ad ogni iterazione si hanno n chiamate al metodo).

Viene utilizzato il metodo [equals\(\)](#) della classe Proiezione per stabilire se la proiezione da modificare coincida con quella letta dal file: in caso affermativo, viene sostituita con la proiezione appena creata (ovvero viene scritta sul file temporaneo); al contrario, viene ricopiata la proiezione dal file sorgente.

Metodo eliminaProiezione()

Il metodo eliminaProiezione() permette di eliminare una stinga (proiezione) all'interno del file "Proiezioni.csv".

Questa funzione è stata implementata basandosi in totale su due file di tipo ".csv": il primo è "Proiezioni.csv" a cui è collegato uno stream per la lettura delle voci (legge riga per riga finché non viene individuata quella da eliminare); il secondo è un file temporaneo (denominato con "pro_numeri.csv", che verrà poi rinominato con il nome del file sorgente, sul quale vengono ricopiate, tramite uno stream collegato in scrittura, le voci dell'altro file a meno di quella interessata.

Al termine della scrittura, il file temporaneo viene rinominato in "Proiezioni.csv" mentre il corrispondente sorgente viene eliminato.

Purtroppo non è stato possibile sviluppare una soluzione più efficace, collegando al codice (tramite stream) un solo file.

Il tutto è inserito in un costrutto "try-catch" che intercetta eventuali eccezioni che si possono sollevare nel caso gli stream non si colleghino al file oppure quest'ultimo è utilizzato dal SO; mentre la clausola "finally" chiude gli stream, sia in caso di avvenuta eliminazione, che in caso di errori, sia esistano prenotazioni per la proiezione da eliminare (verificato con il metodo isEmpty(), che stabilisce se la lista contenente oggetti di tipo "Prenotazione" sia vuota).

All'altezza di ciascun blocco "catch" è stato scelto di stampare a video l'eventuale eccezione che viene sollevata con l'istruzione "System.err.println()", sviluppata appositamente con il campo "err" per mostrare gli errori verificatisi in corso d'esecuzione.

Un ulteriore blocco "try-catch" segue le istruzioni appena descritte: al suo interno vi sono due istruzioni "if", le quali consentono di rinominare il file temporaneo solo se il sorgente viene eliminato.

Nel blocco "try-catch" primario, ovvero le istruzioni atte all'eliminazione, vengono istanziati gli oggetti di tipo "File", creati all'inizio del metodo (quali la rappresentazione astratta del file temporaneo e del sorgente); dopodiché viene richiamato il metodo [getProiezione\(\)](#) per ricercare la proiezione da eliminare con il relativo orario specificato e creati nuovi oggetti (alcuni di tipo "String", un "LocalDateTime" e un "double", oltre al vettore "dati" di tipo "String" per la suddivisione in elementi delle stringhe lette dal file "Proiezioni.csv") che serviranno al [costruttore](#) della classe Proiezione per istanziare un nuovo oggetto di tipo "Proiezione" grazie ai valori di "dati" ottenuti dalle righe del file. Quest'ultimo oggetto è creato ex novo ad ogni iterazione del ciclo "while" e corrisponde alla proiezione contenuta in ciascuna linea del file.

La suddivisione delle stringhe è preferita alla chiamata al metodo getProiezione() per mantenere il tempo di costruzione dell'oggetto lineare (la chiamata ad ogni iterazione del ciclo "while" porterebbe il tempo di computazione da $O(n)$, con n il numero di iterazioni, a $O(n^2)$, in cui ad ogni iterazione si hanno n chiamate al metodo).

È utilizzato infine il metodo [equals\(\)](#) della classe Proiezione per stabilire se la proiezione tratta dal file sia quella da eliminare: in caso contrario, la proiezione letta dal file sorgente viene ricopiata sul temporaneo; al contrario, viene inserita un'interruzione di linea.

Classe cineMax

La classe cineMax contiene il metodo main(), necessario all'esecuzione dell'applicazione.

L'intera classe è composta da una serie di cicli "do-while" e "switch-case" uno annidato nell'altro per gestire il flusso dell'esecuzione a seconda dello specifico utente che utilizza l'applicazione e delle necessità che ha nel corso dell'esecuzione.

Nello specifico, i cicli "do-while" consentono di ripetere un numero indefinito di iterazioni le operazioni che risiedono al loro interno, evitando di interrompersi dopo aver terminato la prima esecuzione. Ad esempio, un utente può avvalersi più volte della funzione di ricerca di una proiezione digitando il numero dell'opzione corrispondente alla stessa che viene riproposto tra le altre opzioni una volta terminato il primo ciclo di ricerca; un altro esempio è dato dalla funzione di login, la quale si ripete più volte finché l'utente non inserisce le credenziali corrette: il metodo infatti ripropone la lettura di username e password finché i corrispettivi valori inseriti non corrispondono a quelli salvati nel file "Utenti.csv"; o ancora, un utente che desidera ricercare una proiezione per un certo criterio di ricerca, ad esempio per titolo del film, può ripetere la ricerca senza dover ogni volta selezionare le scelte che conducono alla funzionalità di ricerca (quindi percorrendo le opzioni del menu dall'avvio alla funzione desiderata) grazie ad un ciclo "do-while" che consente di scegliere tra ricercare, in questo caso, nuovamente per titolo, oppure compiere la ricerca per un altro criterio, e quindi tornare al menu avente a disposizione tutti i criteri.

I costrutti "switch-case" consentono invece di individuare la corretta porzione di codice (un blocco all'interno dell'istruzione "case") a seconda della scelta dell'utente (valore dell'istruzione "switch"), come nel caso della serie di funzionalità disponibili per ogni utente: i clienti possono effettuare una scelta tra prenotare dei biglietti per una proiezione; visualizzare, modificare o eliminare le proprie prenotazioni. In un altro blocco "case" sono presenti le funzionalità disponibili per i biglietti: visualizzare le prenotazioni effettuate per una proiezione.

Nell'ultimo blocco "case", invece, i proiezionisti possono creare, modificare o eliminare una proiezione tra quelle già disponibili nel file "Proiezioni.csv".

Il costrutto appena descritto è inserito come intero blocco "case" all'interno di uno "switch-case" esterno, il quale permette di gestire le funzioni di login, i menu-utente di cui sopra, la possibilità di registrazione di un nuovo utente alla piattaforma e l'opzione di ricerca delle proiezioni, oltre alla possibilità di terminare il programma senza dover ricorrere ai comandi del SO.

Data set di test

L'applicazione si basa su un data set composto da tre file di testo denominati:

- "Utenti.csv" per la gestione (lettura e scrittura) degli utenti registrati;
- "Proiezioni.csv" per la visualizzazione (lato cliente), la modifica e l'eliminazione (esclusivamente per il proiezionista) delle proiezioni;
- "Prenotazioni.csv" per la visualizzazione e la modifica (lato cliente) e sola visualizzazione (lato bigliettaio).

Tutti i file sono memorizzati in una directory denominata "data", sita all'interno del folder radice "cinemax".

Ciascun file memorizza le informazioni sotto forma di stringhe (tramite i metodi toString() delle relative classi), in cui ciascun dato è separato dall'altro dal segno di interpunzione ",".

"Utenti"

Il file "Utenti.csv" è composto da una serie di righe, ciascuna memorizzante un utente.

Ogni utente è riportato sotto una serie di informazioni ordinate sotto il seguente pattern:

"codice_identificativo,nome,cognome,username,password_cifrata,data_nascita,domicilio,ruolo"

"Proiezioni"

Il file "Prenotazioni.csv" è composto da una serie di righe, ciascuna contenente informazioni che compongono una proiezione secondo il seguente pattern:

"data_ora_proiezione,titolo_film,genere,regista,anno_produzione,durata_minuti,eta_minima,prezzo_biglietto"

"Prenotazioni"

Il file "Prenotazioni.csv" è composto da una serie di righe, ciascuna individuante una prenotazione, contenenti le seguenti informazioni sotto il seguente pattern:

"data_ora_proiezione,id_utente,nome,cognome,titolo_film,numero_biglietti_acquistati,codice_prenotazione"

• Limiti della soluzione sviluppata

- L'applicazione non permette a più utenti di avvalersi delle funzionalità contemporaneamente; è consentito perciò l'utilizzo ad un utente per volta.
- Per gli utenti non registrati, la funzionalità di ricerca di una proiezione può avvenire soltanto fornendo all'applicazione un criterio di ricerca alla volta, e non due o più criteri assieme, per il fatto che il metodo [cercaProiezione\(\)](#) è stato implementato per ricevere un solo valore che inizializza uno solo tra i parametri formali (è stato pensato di sviluppare apposta una serie di override del metodo stesso, ciascuno ricevente più valori, ma la soluzione è stata scartata per il fatto di doverne implementare una serie di combinazioni (dal metodo che riceve un solo valore nel primo parametro formale, a quello che ne riceve due, poi tre e così via, tenendo conto delle disposizioni dei valori); inoltre, il titolo è richiesto che venga inserito nella sua interezza; altrimenti, se si ricerca con il titolo parziale, si rischia di non trovare la proiezione.
- Nelle fasi di richiesta di inserimento della data e ora (si veda per es. la ricerca di una proiezione per data e ora) è richiesto di inserire ciò che viene richiesto seguendo il pattern fornito a video, ovvero il formato di data e orario "yyyy-mm-dd hh-mm-ss", altrimenti il programma blocca l'esecuzione e non produce l'output desiderato.
- Le intestazioni dei file ".csv" sono state omesse per il fatto che i metodi, leggendo il documento dalla prima linea, traggono in input valori nel formato "String" (tipo utilizzato per il testo) invece di formati differenti richiesti, per esempio, "LocalDateTime" per la lettura della data e l'ora dal file "Proiezioni.csv".

• Sitografia / bibliografia

Tutorial sulla realizzazione dell'archivio jar: docs.oracle.com/javase/tutorial/deployment/jar/index.html

Repository Github: <https://github.com/Necchio765298/cinemax.git>

Sito web Oracle per download della piattaforma Java:

<https://www.oracle.com/it/java/technologies/downloads/>

Istruzioni su sito web Oracle per installazione:

<https://docs.oracle.com/en/java/javase/26/install/overview-jdk-installation.html>

Testo per realizzazione generale dell'applicazione: "Dai fondamenti agli oggetti: corso di programmazione Java".